Lab 4 – The xv6 Shell from the Outside

Oct 5, 2026 · @Paty

Overview

The whole user interface of xv6 is one program, user/sh.c, about 500 lines long. Today you use it side by side with bash and discover, by experiment, what a shell must do and what bash adds on top. You also write a tiny program that prints the arguments it receives, which shows exactly who expands * and who removes quotes. First, you make sure your sleep program from Lab 2 builds on the current xv6.

Course: Operating Systems, FCIM / FAF, UTM, 2026-2027

Lab 4: The xv6 Shell from the Outside (Track B: build your own small OS, xv6)

Duration: 90 minutes

General requirements (for every lab)

1. Word report (.docx) following the UTM structure

- UTM title page: university, faculty, department; “Report on laboratory work no. 4”, course, topic; “Done by: st. gr. ___, Name Surname”, “Checked by: ___”; Chișinău, 2026.
- Content: purpose of the work, the work done part by part (the comparison table, outputs, answers to every **Observe** box), **conclusions**.
- Every screenshot is centered, with a centered caption below it: **Figure 1.1 – Name of the figure** (first number = the part, second = the figure number within that part).
- File name: Lab4_NameSurname_Group.docx.

2. Git repository

- Use the same repository as for Lab 3 (**os-2026-NameSurname**), with a new folder **lab4/** (the .docx report, sleep.c, args.c, args_linux.c, Makefile), uploaded with:

```
git add lab4 && git commit -m "Lab 4" && git push
```

Before you start (~5 min)

Check that xv6 builds and boots on your laptop:

```
bash os2026-setup.sh --check
```

If the check fails, run the script without `--check`. It takes 5 to 10 minutes, installs everything and downloads xv6 into ~/xv6-riscv; your existing work there is never touched.

```
bash os2026-setup.sh      # only if the check failed
```

Keep a notes file open while you work; you will build the Word report from it.

Part 1. Fix sleep for the current xv6 (~15 min)

In August 2025 the xv6 authors renamed the system call `sleep()` to `pause()`. Your sleep program from Lab 2 may no longer build.

What to do:

1. Check which name your tree uses:

```
cd ~/xv6-riscv
grep -n "pause\|sleep" user/user.h
```

2. If user.h declares `pause`, change the call in your user/sleep.c as below. Make sure `$U/_sleep\` is in UPROGS in the Makefile.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // was sleep() before Aug 2025
  exit(0);
}
```

3. Build, boot and test:

```
make qemu
$ sleep 20
$ sleep
usage: sleep <ticks>
```

Makefile reminder: each UPROGS line starts with a TAB and ends with a backslash. Nothing may follow the backslash, not even a comment. Exit QEMU with **Ctrl-A**, then **X**.

Minimum screenshots: Figure 1.1 – The grep result in user.h; Figure 1.2 – The updated sleep.c; Figure 1.3 – sleep running in xv6, with and without an argument.

Part 2. bash versus the xv6 shell (~30 min)

What to do: run every command below twice:

- once in **bash** on Linux, in an empty test folder that contains a file named README (`mkdir ~/shtest && cd ~/shtest && touch README`);
- once at the **xv6 $ prompt** (`make qemu`).

Fill in the table in your report: what each shell printed, and why.

Command	Tests	bash	xv6
echo hello > f ; cat < f	redirection		
ls | grep READ	pipe		
(echo a; echo b) | wc	grouping, list		
echo one ; echo two	sequence		
echo bg &	background		
ls *	globbing		
echo $HOME	variables		
echo "a b"	quotes		
mkdir d ; cd d ; ls ..	PATH		
arrow up, Tab, Ctrl+R	history, completion		

Observe (answer in the report):

1. List the features both shells have, and the ones only bash has.
2. In xv6, ls stops working after cd d. Why? Find a way to run ls from inside d anyway. (Hint: where do the xv6 programs live?)
3. Which missing feature would you miss most in daily work, and which would be easiest to add?

Minimum screenshots: Figure 2.1 – The commands run in bash; Figure 2.2 – The same commands run in xv6; Figure 2.3 – ls failing after cd d, and the working alternative.

Part 3. Who expands *? A program that prints its argv (~25 min)

What to do:

1. Write **user/args.c** for xv6 and add `$U/_args\` to UPROGS:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

2. Write the same program for Linux as **args_linux.c**: include `<stdio.h>` instead of the two xv6 headers, and use `return 0` instead of `exit(0)`. Compile it:

```
gcc -o args args_linux.c
```

3. Run the SAME lines in both shells (in bash, use `./args` and run it in a folder that has some .c files):

```
args one   two   three
args *.c "x y"
args $HOME
args 'single quoted'
```

Observe (answer in the report):

1. Paste both outputs for `args *.c "x y"`. How many arguments did each program receive?
2. So who turns *.c into file names and removes the quotes: the program, the kernel, or the shell?
3. What happened to the extra spaces in `one two three`?

Minimum screenshots: Figure 3.1 – args.c and the UPROGS line; Figure 3.2 – args output in xv6; Figure 3.3 – args_linux output in bash.

Part 4. Read the shell (~15 min)

What to do: inside ~/xv6-riscv, run the commands below and read the parts of user/sh.c they point to.

```
wc -l user/sh.c
grep -n "#define" user/sh.c      # the kinds of commands
grep -n "case " user/sh.c        # where each kind is run
grep -n "cd" user/sh.c | head
```

Observe (answer in the report):

1. Name the five kinds of command the xv6 shell knows and the character that produces each.
2. Where is cd handled, and why is it not run as a separate program?
3. Search sh.c for any code that handles * or $. What does that explain about Part 2?

Minimum screenshots: Figure 4.1 – The command kinds defined in sh.c; Figure 4.2 – Where each kind is run; Figure 4.3 – The code that handles cd.

Deliverables and grading

In the **lab4/** folder of your Git repository: the report Lab4_NameSurname_Group.docx, user/sleep.c (working on your tree), user/args.c, args_linux.c and the Makefile with your UPROGS lines. Copy them from ~/xv6-riscv:

```
mkdir -p ~/os-2026-NameSurname/lab4
cp ~/xv6-riscv/user/sleep.c ~/xv6-riscv/user/args.c ~/xv6-riscv/Makefile
~/os-2026-NameSurname/lab4/
cp args_linux.c ~/os-2026-NameSurname/lab4/
```